Day 1 Live-Coding Session — 5 Problems

---

1. Rock-Paper-Scissors Game

Scenario

The College Coding Arcade

The coding club is setting up a mini arcade module for orientation day, where new students play a quick Rock-Paper-Scissors match against the computer as a warm-up before their placement round. The club needs a simulator that plays several rounds between the "player" and the "computer," records the outcome of every round, and prints a final scoreboard summarizing wins, losses, draws, and the player's win percentage.

Task

- Generate the computer's move randomly (Rock, Paper, or Scissors) for each round.
- Accept the player's move for each round (via input, or a predefined list for a live demo).
- Determine and display the winner of each round using standard Rock-Paper-Scissors rules.
- Repeat for N rounds (suggested N = 5).
- After all rounds, print a summary table: Round | Player Move | Computer Move | Result.
- Print the total Wins, Losses, Draws, and the player's Win Percentage.

Suggested method signature(s): String playRound(String playerMove, String computerMove)

Sample Input / Output

Input	Output
Round 1 — Player: Rock, Computer: Scissors	*Player Wins*
Round 2 — Player: Paper, Computer: Paper	*Draw*
Round 3 — Player: Scissors, Computer: Rock	*Computer Wins*
Final Summary (after 5 rounds)	*Wins: 2 | Losses: 2 | Draws: 1 | Win % = 40.0%*

Concepts covered: Random number generation, conditional logic, loops, arrays for the round table, formatted/tabular output, basic statistics (percentage calculation).

2. Palindrome Checker (3 Approaches)

Scenario

The QA Text Verification Toolkit

The QA team on a coding-assessment platform is building a small internal toolkit to sanity-check palindrome-detection logic before it ships to students as a graded exercise. To make sure the logic is solid regardless of implementation style, the toolkit must verify a given text using three independent approaches — iterative comparison, recursion, and array reversal — and confirm all three agree on the result.

Task

- Accept a text input (e.g., a word or short phrase).
- Implement an iterative check: compare characters from both ends moving toward the middle.
- Implement a recursive check: recursively compare the first and last characters, shrinking the substring each call.
- Implement an array-reversal check: convert the string to a character array, reverse it, and compare it to the original.
- Print the result of all three approaches for the same input — they should always agree.

Suggested method signature(s): boolean isPalindromeIterative(String text) + boolean isPalindromeRecursive(String text) + boolean isPalindromeArrayReversal(String text)

Sample Input / Output

Input	Output
"madam"	*Iterative: Palindrome | Recursive: Palindrome | Array Reversal: Palindrome*
"hello"	*Iterative: Not Palindrome | Recursive: Not Palindrome | Array Reversal: Not Palindrome*

Concepts covered: Loops, recursion, array manipulation, string comparison, comparing implementation trade-offs.

3. BMI Calculator for a Team

Scenario

The Corporate Wellness Program

An organization's HR wellness team is running a health check-up camp for a department of employees. For each employee, height (in meters) and weight (in kg) are recorded, and the system must calculate BMI and classify their health status, then present all details in a clean table for the wellness report.

Task

- Take height (m) and weight (kg) for a team of people (suggested: 10; use arrays, and random values for a fast live demo).
- Compute BMI = weight / (height × height).
- Classify status: BMI < 18.5 → Underweight; 18.5–24.9 → Normal; 25–29.9 → Overweight; ≥30 → Obese.
- Display a table: Person | Height (m) | Weight (kg) | BMI | Status.

Suggested method signature(s): String getBmiStatus(double bmi) + void printWellnessReport(double[] heights, double[] weights)

Sample Input / Output

Input	Output
Person 1 — Height: 1.75 m, Weight: 70 kg	*BMI: 22.86 | Status: Normal*
Person 2 — Height: 1.60 m, Weight: 90 kg	*BMI: 35.16 | Status: Obese*

Concepts covered: Parallel/2D arrays, arithmetic operations, conditional logic, formatted tabular output.

4. First Non-Repeating Character

Scenario

The Unique Letter Hunt Mini-Game

A code-learning platform runs a light-hearted "Unique Letter Hunt" mini-game between exercises: a player types any word or sentence, and the app must instantly find and highlight the first character that appears only once in the entire input — a fun, quick way to reinforce character-frequency thinking between coding drills.

Task

- Accept an input string (a word or short sentence).
- Compute the frequency of every character in the string.
- Scan the string left to right and find the first character whose frequency is exactly 1.
- Print that character, or a clear message if no non-repeating character exists.

Suggested method signature(s): char findFirstNonRepeatingChar(String text)

Sample Input / Output

Input	Output
"swiss"	*First Non-Repeating Character: 'w'*
"aabbcc"	*No Non-Repeating Character Found*

Concepts covered: Character frequency counting, loops, HashMap/array-based counting, early-exit scanning.

5. Reverse Customer Name

Scenario

The Customer Identity Verification System

You are developing a customer identity verification module for a banking application. As part of an internal security exercise, the system generates a reversed version of a customer's name to verify whether the application correctly processes character sequences without modifying the original data. This feature is used only for testing and training purposes. The system should accept a customer's name and display its reverse while keeping the original name unchanged.

Task

- Develop a method that reverses the given customer name.
- The method takes one input argument: customerName.
- It should return the name in reverse order.
- Call this method from main and print both the original and reversed names.

Suggested method signature(s): String reverseCustomerName(String customerName)

Sample Input / Output

Input	Output
"Sunil"	Original Name: Sunil Reversed Name: linuS

Concepts covered: String traversal, character array manipulation, string reconstruction.